Task Manager Project Roadmap

Version 1: Basic Console Task Manager

Goal: Build the simplest working task manager.

Features:

- Add task
- View tasks
- Complete task
- Delete task
- Exit program
- Tasks only exist while program is running

Skills practiced:

- Classes
- Objects
- ArrayList
- Methods
- Loops
- Scanner
- Basic OOP
- Git commits

Ready to move on when:

- All V1 features work
- Invalid inputs do not crash the program
- Code is split into Main, TaskManager, and Task
- You completed the testing checklist
- You committed a working V1 to GitHub

Version 2: Input Validation and Better Error Handling

Goal: Make the program harder to break.

Features:

- Prevent crashes from letters when numbers are expected
- Handle empty task names
- Handle invalid task numbers
- Handle viewing empty list

- Handle completing/deleting when no tasks exist
- Keep asking until valid input is entered

Skills practiced:

- Defensive programming
- Scanner issues
- Loops
- Boolean flags
- Edge cases
- Cleaner helper methods

Ready to move on when:

- User cannot easily crash the program
- Every bad input gives a useful message
- Input validation code is not repeated everywhere
- Main still does not become messy

Version 3: Save and Load Tasks From File

Goal: Tasks should still exist after closing the program.

Features:

- Save tasks to a file
- Load tasks when program starts
- Save task name and completion status
- File format could be simple text or CSV

Example file: Study Java,false Go To Gym,true

Skills practiced:

- File reading
- File writing
- Parsing strings
- Data persistence
- Handling missing files

Ready to move on when:

- Tasks load correctly when program starts
- Tasks save correctly when program exits
- Completed status is preserved
- Program does not crash if file does not exist

Version 4: Rename and Mark Incomplete

Goal: Use methods you already planned but did not fully expose in V1.

Features:

- Rename task from menu
- Mark task incomplete
- Updated menu options

Skills practiced:

- Adding features to existing code
- Reusing getTaskByNumber()
- Expanding without breaking old features

Ready to move on when:

- Rename works
- Mark incomplete works
- Old features still work
- Menu is still readable

Version 5: Task Details

Goal: Make tasks more realistic.

Add fields:

- Description
- Priority
- Due date
- Category

Possible features:

- Add task with priority
- View full task details
- Edit task details

Skills practiced:

- Expanding a class
- Constructor design
- Method overloads

- Managing more data
- Keeping code organized

Ready to move on when:

- Task class is still understandable
- TaskManager is not overloaded with too much logic
- You can explain every new field and why it exists

Version 6: Search, Filter, and Sort

Goal: Make the task manager more useful.

Features:

- Search tasks by name
- Show only completed tasks
- Show only incomplete tasks
- Sort by priority
- Sort by due date

Skills practiced:

- Loops
- Searching
- Filtering
- Sorting
- ArrayList manipulation
- Comparing objects

Ready to move on when:

- Search works
- Filters work
- Sorting works
- You understand the difference between search, filter, and sort

---

Version 7: Refactor Into Cleaner Architecture

Goal: Clean up the project like a real developer.

Possible changes:

- Create a Menu class
- Create a FileManager class

- Create helper methods
- Remove duplicated code
- Improve method names
- Improve error handling

Skills practiced:

- Refactoring
- Separation of concerns
- Reading old code
- Improving design without changing behavior

Ready to move on when:

- Code is easier to read than before
- Main is not too large
- Each class has a clear job
- No feature broke during refactoring

Version 8: Portfolio Polish

Goal: Make the project presentable on GitHub.

Add:

- README
- Project description
- Features list
- How to run
- Version history
- Screenshots of console output
- What you learned
- Future improvements

Skills practiced:

- Documentation
- GitHub presentation
- Explaining projects
- Resume preparation

Ready to move on when:

- A stranger can understand the project from the README
- GitHub repo looks clean
- You can explain the project in an interview

Version 9: GUI Version

Goal: Turn the console app into a visual app.

Possible options:

- JavaFX
- Swing

Features:

- Buttons
- Text fields
- Task list display
- Complete/delete buttons

Skills practiced:

- Event-driven programming
- GUI layout
- Separating logic from interface
- Reusing Task and TaskManager

Ready to move on when:

- GUI works without rewriting all logic
- Task and TaskManager still work independently
- You understand how the GUI talks to the backend classes

Version 10: Database Version

Goal: Store tasks in a real database.

Possible database:

- SQLite

Features:

- Save tasks in database
- Load tasks from database
- Update completed status
- Delete tasks from database

Skills practiced:

- SQL
- Databases
- Persistence
- CRUD operations
- Real-world app structure

Ready to move on when:

- You can add, view, update, and delete tasks from the database
- You understand basic SQL
- You can explain what CRUD means

Recommended Path

Do not try to build all versions quickly.

Recommended order:

1. Version 1: Basic console app
2. Version 2: Better input validation
3. Version 3: Save/load from file
4. Version 4: Rename and mark incomplete
5. Version 6: Search/filter/sort
6. Version 8: Portfolio polish

Versions 5, 7, 9, and 10 can come later depending on your growth.

Rule for Moving Versions

Only move to the next version when:

1. Current version works
2. Current version is tested
3. Current version is committed
4. You can explain how it works
5. You are not carrying major bugs forward

Do not upgrade broken code.

Fix first. Commit. Then expand.